ASIoP Summer Project Research Final Report

Yueh Hsuan Huang

July ∼ August 2024

Contents

In this file, I would talk about what I did in the summer program at ASIoP, in Jul-Aug, 2024. If there is anything unclear, feel free to contact me here : b11901099@ntu.edu.tw Also, the code I use (the src_new folder) is in Dropbox and there is a read_me.txt in it.

1 Introduction

In this part I would briefly discuss why people are interested in tensor networks and why researchers would like to use tensor networks on machine learning problems.

1.1 Why tensor network?

Tensor Networks is a system composed of multiple tensors and has applications in many physics fields. Its power in decomposing complex systems and bringing good-enough approximation has led researchers to implement it in fields such as quantum physics, quantum computing, statistical physics, and many other fields. Also, different structure could possibly capture different traits and hidden relation of a specific problem. There are algorithms for specific type of tensor network structures. Hence, deciding which structure to use is also important when using tensor network.

1.2 Why do we care about the tensor network? What could it bring (possible benefits)?

We now know that tensor networks are intrinsically capable of decomposing complex systems, but how do tensor networks achieve that? One concept is using tensor networks to represent functions. This idea (or discovery) is powerful in the sense that a lot of the physics systems that we've encountered are governed by formulas and equations. Another concept is to use different structure of tensor networks to capture the correlations in the system. For example, matrix product states(MPS) or tree tensor network are different structure to analyze a system.

1.3 Motives of this project

An intuitive thought would be like this: applying tensor network on the system we are familiar with and seeing if there would be some benefits. In my project, I focus on how researchers use tensor networks on machine learning problems, specifically supervised learning problems. Even though researchers have already developed many algorithms and structures to solve supervised learning problems, the interpretation of what has the model learned remain a difficult task. We use tensor networks here not only because it has well-developed algorithms, but also because we want to understand the essence of one single problem by looking into the tensor network and gain some insights. Since tensor network is totally linear, it is natural to assume it is easier to interpret it.

2 The problem

2.1 Supervised learning

In machine learning problems, the goal is to use mathematical model to imitate the unknown rule that governs the dataset. Supervised learning problems are machine learning problems with labeled data. Labeled data usually has more for the model to learn, and are nice to test the ability of the model and algorithm.

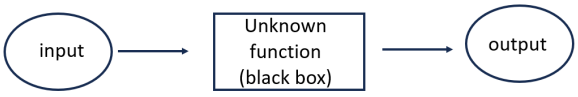


Figure 1: Machine learning flow chart

2.2 Tasks here

First of all, I want to try the procedure on MNIST dataset. MNIST dataset is a widely used dataset for benchmarking in the machine learning research area. It could be used to check the efficiency of the capability of a model. MNIST dataset has images of handwritten digits. All these images are in greyscale and the size is 28x28, which means that there would be 784 pixels for every image. The default number of training examples is 60000, and for testing, it's 10000. The label is a scalar. For example, an image that has 6 in it would have 6 as its label. In Mile's paper, they have used a binary encoding of the label. For instance, the label 6 would be [0,0,0,0,0,0,1,0,0,0] in Mile's encoding.
I first tried Mile's original code, but it did not work out. This code is here, and is in C++.

Figure 2: MNIST dataset

Then, I tried another code found online, but it also did not work out on MNIST dataset. Therefore, I decide to test my code on less complex dataset. The tasks here is finding max and the data looks like this :

$$[0.5\ \ 0.4\ \ 0.6\ \ 0.2\ \ 0.3] \rightarrow [0\ \ 0\ \ 1\ \ 0\ \ 0]$$

Figure 3: Data of finding max

The vector on the left is the input data, and the vector on the right is the label. Every entry is a randomly generated number between 0 and 1. The label vector would be the same size as the input vector, and the 1 signify the largest-value entry.

3 Model

3.1 MPS and weight matrix

In this problem, the power of tensor networks comes in at the MPS representation of the weight matrix.

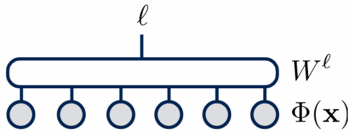


Figure 4: Weight and the input data

We follow the design as what previous researchers has done, and Figure 4 is a simple diagram that describe it. The equation that describe the figure is :

$$f^l(x) = W \cdot \Phi(x)$$

$f^l(x)$ is the output prediction. l could go from 0 to the number of possible predictions. For example, in the MNIST dataset, l could be 0, 1,..., 9. For finding max task with 5 entries, l could be 0, 1, 2, 3, 4
Then, we use MPS form to simplify the big weight tensor.

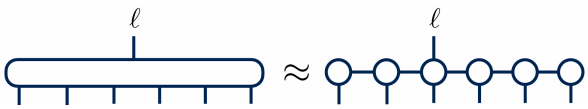


Figure 5: MPS to approximate weight tensor

The dimension between tensors is called bond dimension, and the dimension dangling out is called visible index. This move would greatly reduce the space requirement for storing the weight. I'll try to explain this from a technical point of view. Let's say the dimension of the visible index of the big tensor is d. Assume the big tensor has only N visible index and no bond index. Then the number of all the parameter in this big tensor is d^N. On the other hand, in the MPS, let's say dimension of the visible index is also d, and the bond dimensions are all m. Then, the MPS form would have Nm^2d parameters, which is a huge shrink from the big tensor form.

4 Procedure

4.1 Preprocess the dataset

For image data, I would flatten the image into a row vector by using .flat() in python. For the finding max task, there is no need to flatten the data. After flattening the data, feature mapping would be applied onto the flattened row vectors. The feature mapping used here looks like this :

$$\phi^{s_j}(x_j) = \left[\cos\left(\frac{\pi}{2}x_j\right),\ \sin\left(\frac{\pi}{2}x_j\right)\right]$$

Figure 6: Feature mapping

x_j is the j-th entry in the row vector data. In the MNIST case, it would be normalized by dividing by 255, since greyscale pixels would range from 0 to 255. For finding max task, the entry would already be between 0 and 1. s_j is the dangling index that can be either 0 or 1. This feature mapping is normalized and could possibly bring some insight to the problem. The reason of using feature mapping is that what feature map does is mapping input to a higher dimensional space. More intuitively, this means that new features are added for the model to learn from. Sometimes, this approach could help and sometimes it could not.

What's the role of feature mapping : In "Kernel learning", the kernel mapping maps the original data sets to a higher dimensional space. This mapping seems redundant, but just treat x as the compressed data, $\phi(x)$ as the original data, that would make more sense. Just like "augmenting the data with more features".
(One example might be like this: you try to predict the price of a piece of land. Intuitively thinking, the price would be proportional to the land's area. However, you only have information about the width and length of the land. So you add the feature "area" into your data. This is a way of feature mapping. Adding extra feature is like mapping data into a higher dimensional space. Check out "feature engineering" for more information)

4.2 Initialization of the weight matrix

Finding a good starting point for training has always been an important topic for ML problems. Usually, in ML applications, how to initialize weight would depend on the task and the training algorithm. However, in here, not much has been explored about initializing an MPS. Therefore, there are 2 main methods to choose for initialization. One is use representer theorem. This method initialize the weight by summing up mapped input data. Another one is constant initialization. Every entry in the weight matrix would be a constant that can be tuned. Here, I use constant initialization.

4.3 Cost function and feature map

The cost function used here is the quadratic cost function :

$$Cost = \frac{1}{2}\sum_{n=1}^{N_T}\sum_{l}(f^l(x_n) - \delta^l_{L_n})^2$$

$\delta^l_{L_n}$ is the label. Here, what the cost function does is measure the "distance between the unknown function and the function we have now". Also, this quadratic form ensures that we could use conjugate gradient in the next step. It is also possible to try other type of cost function such as the cross-entropy cost function that is popular for classification problem.

4.4 Training of the weight matrix

There are multiple steps in the training session. The main algorithm is inspired by DMRG (density matrix renormalization group) in tensor networks.

1. Contracting neighboring tensors : the training start with contracting two neighboring tensors. We call this contracted tensor "bond tensor". By doing contraction, DMRG could adaptively change the bond dimension between neighboring tensor.

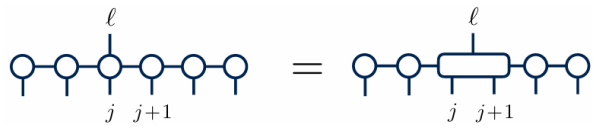


Figure 7: Contracting tensors

2. Contract other tensors

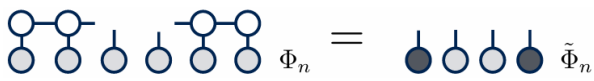


Figure 8: Contract other tensors

3. Update the bond tensor : Use conjugate gradient or gradient descent on the cost function with respect to the bond tensor. Then, we could update the bond tensor.

4. SVD (single value decomposition) and put back the updated tensor : After updating, use SVD to cut the contracted tensor back to the form before contracting them together.

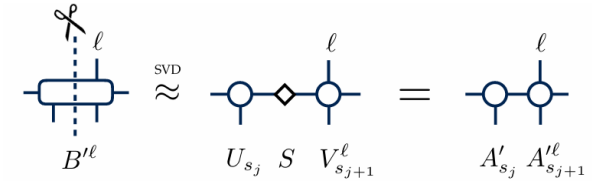


Figure 9: SVD to split up the contracted bond tensor

There is no limitation on the bond dimension when performing SVD. Another way of updating is to fix the bond always as a specific number. A comparison on that would be nice. After putting the bond tensors back into the weight matrix, we would shift one tensor and then move on to contract next two tensors.

In one sweep, we would first focus on the leftmost tensors. We would shift one tensors at a time until we reach the rightmost tensor. Then, we would move toward left and shift one tensor at a time all the way back to the leftmost tensor. We perform the steps above iteratively in one sweep. Ideally,

the sweep would stop after convergence. Another way to stop training is to set a threshold of the difference of error between sweeps. The following plot shows the evolution of error. The y-axis is the error in log scale. The x-axis is the number of sweeps. It can be seen the error converge nicely at the end of sweeps.

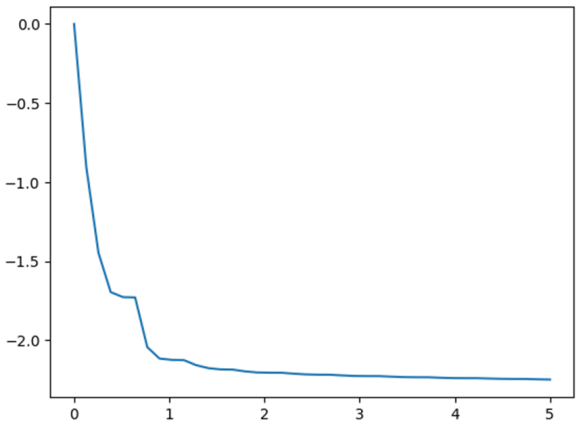


Figure 10: 100 examples, 5 tensors, 5 sweeps

4.5 Calculate the accuracy

After training, take out the testing set and put it use the weight matrix to calculate accuracy. The way we interpret the prediction is that we would take the max position as 1 and others as zero. For example, a prediction like this [0.2 0.2 0.8 0.2] would be interpreted as "predicting the max to be the third element". And if the max is really at the third position, we say it is correctly predicted, otherwise, the prediction is incorrect. And accuracy = (# correct in training) / (# training).

5 Results and Discussion

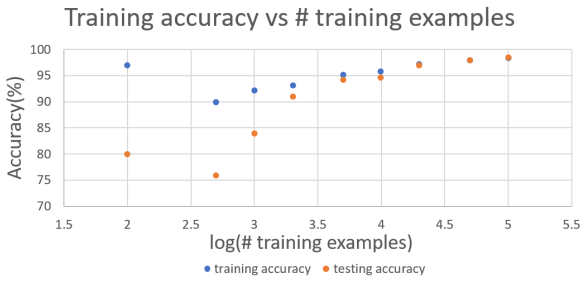
5.1 Results

5.1.1 5 elements finding max

5 sweeps, start with bond = 2

	training accuracy	testing accuracy
100	97%	80%
500	90%	76%
1000	92.2%	84%
2000	93.15%	91%
5000	95.22%	94.2%
10000	95.77%	94.7%
20000	97.245%	97%
50000	97.974%	98.08%
100000	98.306%	98.44%

(a) Accuracy of 5 features vs number of training examples 1



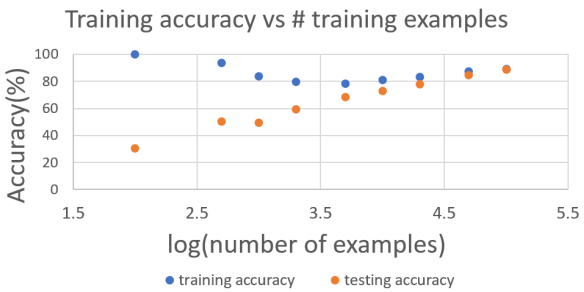
(b) Accuracy of 5 features vs number of training examples 2

5.1.2 10 elements finding max

10 sweeps, start with bond = 2

	training accuracy	testing accuracy
100	97%	80%
500	90%	76%
1000	92.2%	84%
2000	93.15%	91%
5000	95.22%	94.2%
10000	95.77%	94.7%
20000	97.245%	97%
50000	97.974%	98.08%
100000	98.306%	98.44%

(a) Accuracy of 10 features vs number of training examples 1



(b) Accuracy of 10 features vs number of training examples 2

5.1.3 Number of parameters vs number of tensors

I try to check the number of parameters in the trained weight. The following example figure shows how I do that.

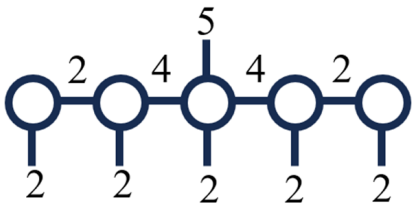


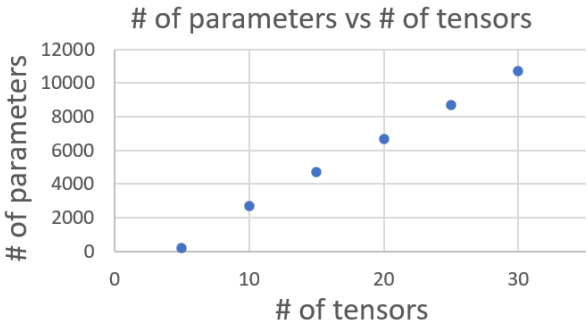
Figure 13: Bond in weight

There are 5 tensors here. The leftmost tensor : 2x2 parameters. The second tensor : 2x2x4. The third tensor : 4x4x2x5. The fourth tensor : 2x2x4. The fifth tensor : 2x2.

This plot shows the comparison between size of input vector(# of tensors) and number of parameters.

size of vector	# of parameters
5	200
10	2688
15	4688
20	6688
25	8688
30	10688

(a) Accuracy of 10 features vs number of training examples 1



(b) Accuracy of 10 features vs number of training examples 2

I also use log-log plot to check the relationship between # of parameters and # of tensors.

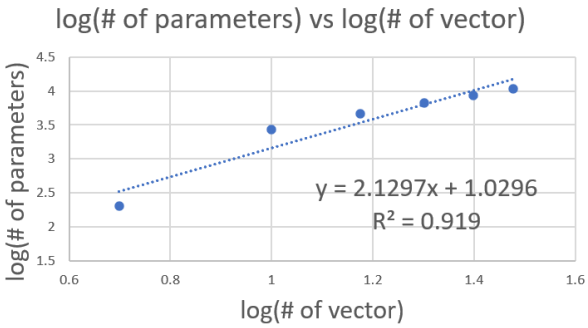


Figure 15: log-log scale check

The slope here is around 2.13, and the R^2 is around 0.92. The R^2 value is okay, and the slope shows that the number of parameters roughly grows linearly with the square of number of vectors.

5.2 Other discovery

I found that the training time is roughly linear with the number of training examples.

5.3 Discussion

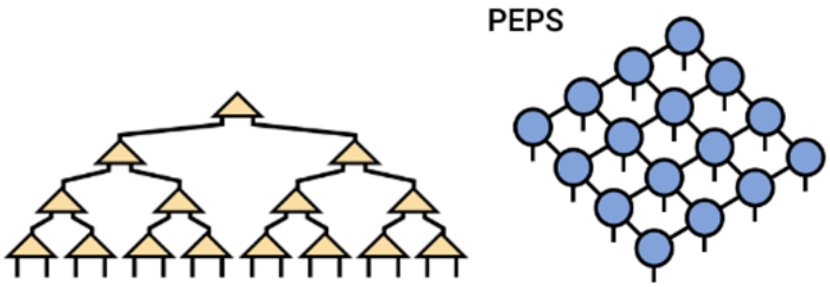
1. More training example, lower training accuracy and higher testing accuracy. My interpretation of this is that more example for training, less possibility for overfitting. When overfitting, the model would get too close to the training set, which would result in high training accuracy.

2. In here, we discuss the " finding max " type of problem. This problem requires a global knowledge of the input vector to answer correctly. However, MPS is known for its ability to track local correlation between neighboring or close sites and these information transmitted in between might be nice for us to learn about the whole system. Therefore, it is unclear whether we should use MPS to solve this " finding max " type od problem. An intuitive method would be using tensor tree to solve this kind of global information problem. Even though the number of comparison need to be made is the same as naively sorting and shifting the max toward one end of the tensor MPS. We would expect the bond dimension be the same for every bond in the tensor tree.

6 Future direction

I use bullet points to discuss some possible future direction.

1. Different tensor networks structure : so now for finding max tasks, using tree tensor network. Also, Miles mentioned that PEPS could possibly be great for analyzing 2D image data. This paper has discuss PEPS on MNIST.



(a) Tree tensor network (b) PEPS

2. Different machine learning problem : for example, the problem I tried here is a classification problem. So the label has to be in binary encoding. However, there is another kind of problem called regression. Maybe in the future, tensor network could be applied on regression type of problem.

   So this direction can have multiple steps. First, I can try this algorithm on different classification dataset like the fashion MNIST and CIFAR-10. These dataset are still in the realm of classification problem but trying MPS on these can tell us the limit of this MPS and the algorithm. Then, after testing out the power of our algorithm and if it works, then we can move on to regression type of problem.

References

[1] Tensor network for machine learning applications 1 (YouTube link)

[2] E. M. Stoudenmire, D. J. Schwab, Supervised Learning with Quantum-Inspired Tensor Networks, Adv. Neu. Inf. Proc. Sys. 29, 4799 (2016), arXiv:1605.05775

[3] Rom´an Or´us, *A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States.* arXiv:1306.2164, Jun. 2014.

[4] V. Bertret, PIR-Tensor-Network-MNIST, https://github.com/vbertret/PIR-Tensor Network MNIST (2021)

[5] E. M. Stoudenmire, TNML https://github.com/emstoudenmire/TNML

[6] Jacob C Bridgeman and Christopher T Chubb (2017) *Hand-waving and interpretive dance: an introductory course on tensor networks.* J. Phys. A: Math. Theor. 50 223001